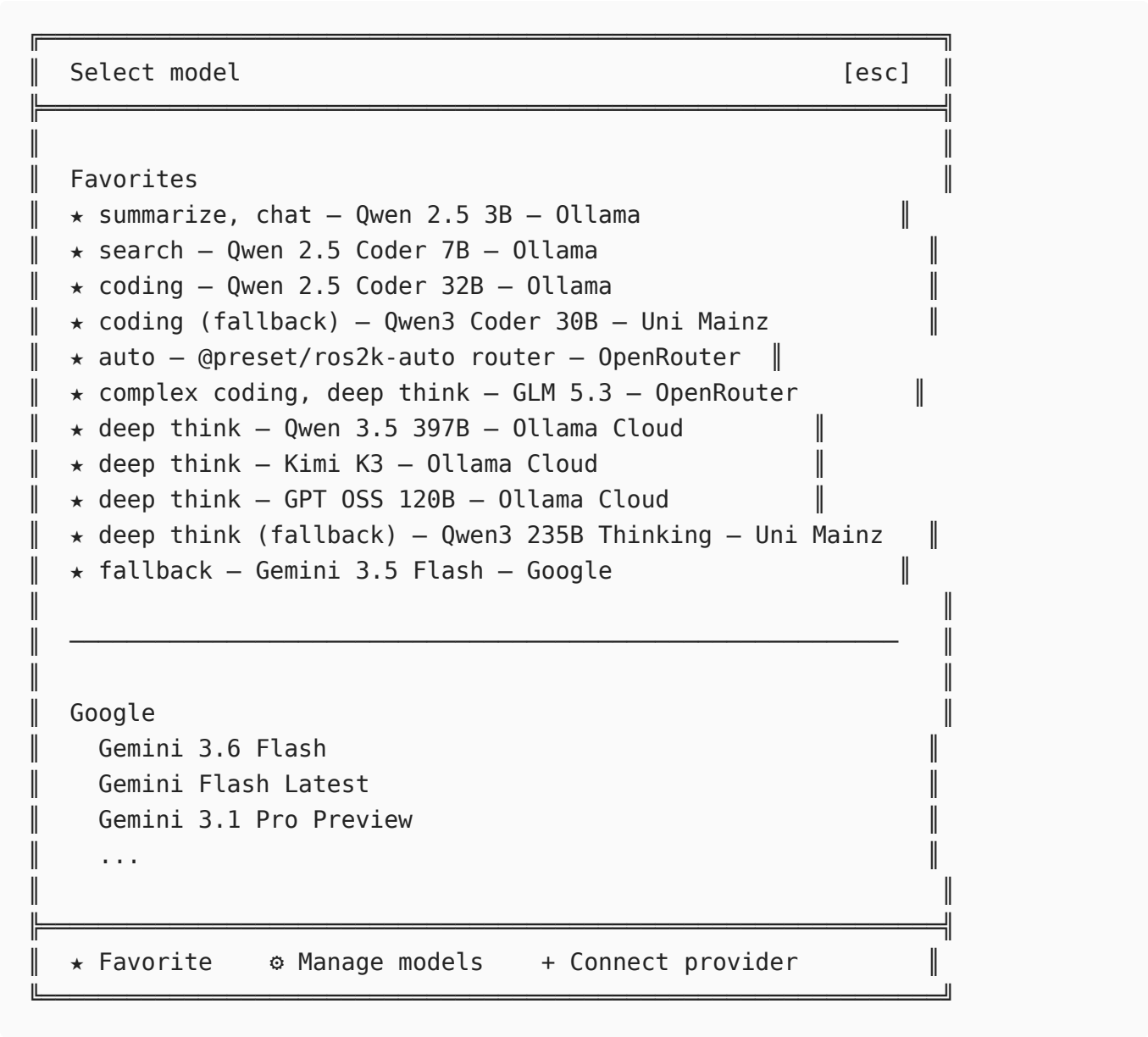


Model Selection Strategy for ROS2K Development

Version: 1.3 | Date: 2026-08-25 | Status: Draft for review

Favorites Preview

This is what the opencode `/models` picker will look like after rollout:



The 11 starred entries are pre-populated via `~/.local/state/opencode/model.json` (XDG **state** dir — NOT `~/.local/share/opencode/` ; see section 5.5).
Team members get the same list on first launch (minus the 3 Ollama Cloud entries

if they don't have a key — see section 2.3). Below the Favorites, all non-favorited models appear grouped by provider. See section 5.5 for details.

Note: the "auto" entry is the OpenRouter preset `@preset/ros2k-auto`. When selected, `opencode` sends `model: "@preset/ros2k-auto"` to the OpenRouter API, which resolves the preset server-side: the Auto Router classifies the prompt and selects the best free model from the 5-model whitelist. The response shows which model was actually used. See section 2.4 for details.

1. Problem Statement

ROS2K development spans tasks from trivial (session titles) to complex (race-condition analysis in concurrent tmpfs file polling). A single LLM cannot optimally serve all task types. Blind token spending on OpenRouter consumed \$199.53 of \$300 budget with no task-aware routing.

This document defines a task-complexity-graded model selection strategy that minimizes cost while maximizing quality and latency fit across six providers, two development environments, and ten team members.

2. Infrastructure

2.1 Providers

Provider	Access	Cost model	Rate limit	Team scope
Ollama (local)	Per-laptop GPU (24 GB / 16 GB / 12 GB)	Free (electricity)	GPU contention with Gazebo	Per-laptop
OpenRouter	API key, workspace <code>ba22b3ae</code>	Pay-per-token (\$100.47 remaining)	None (paid)	Workspace presets shared
Google AI	API key (free tier)	Free (rate-limited)	RPD quota per key	Per-key
Uni Mainz	API key (university)	Free	20 req / ~5 min window	Shared key
Ollama Cloud	API key (shared)	\$10/month shared budget	Unknown	All team members
GitHub Copilot	VSCoide + CLI, academic quota	Free (faculty)	GitHub-managed	All 10 members

2.2 Local GPU Budget

2.2.1 Primary Laptop (RTX 5090, 24 GB VRAM)

Model	VRAM	Load time	Notes
qwen2.5-coder:32b (Q4_K_M)	~19 GB	~3s	Code-specialized, 32K context, tool-calling
qwen2.5-coder:7b (Q4_K_M)	~4.7 GB	~1s	Code-specialized, 32K context, tool-calling
qwen2.5:3b (Q4_K_M)	~1.9 GB	~0.5s	General-purpose, 32K context, tool-calling

Ollama loads one model at a time. VRAM is shared with Gazebo (ROS2K simulation) — when Gazebo runs, local LLM inference competes for GPU and latency degrades.

2.2.2 Weaker GPU Variants (RTX 5080 16 GB, RTX 4080 12 GB)

Not all team members have a 5090. The 32B coder (the "coding" primary) needs ~19 GB VRAM and does not fit on 16 GB or 12 GB GPUs. The strategy adapts automatically: weaker-GPU members use cloud models for the "coding" mnemonic while keeping local 3B/7B models for background tasks.

GPU	VRAM	32B fits?	7B fits?	3B fits?	Coding model
RTX 5090 Laptop	24 GB	yes	yes	yes	ollama/qwen2.5-coder:32b (local)
RTX 5080	16 GB	no	yes	yes	uni-mainz/Qwen3 Coder 30B (cloud, free)
RTX 4080 Laptop	12 GB	no	yes	yes	uni-mainz/Qwen3 Coder 30B (cloud, free)

Impact per mnemonic on weaker GPUs:

Mnemonic	5090 (24 GB)	5080 (16 GB) / 4080 (12 GB)	Impact
summarize	ollama/qwen2.5:3b	ollama/qwen2.5:3b	None — 3B fits everywhere

Mnemonic	5090 (24 GB)	5080 (16 GB) / 4080 (12 GB)	Impact
chat	ollama/qwen2.5:3b	ollama/qwen2.5:3b	None
search	ollama/qwen2.5-coder:7b	ollama/qwen2.5-coder:7b	None — 7B fits everywhere
coding	ollama/qwen2.5-coder:32b	uni-mainz/Qwen3 Coder 30B (cloud)	Major — no local 32B
complex coding	openrouter/z-ai/glm-5.3	openrouter/z-ai/glm-5.3	None — cloud-based
deep think	openrouter/z-ai/glm-5.3	openrouter/z-ai/glm-5.3	None — cloud-based

Only the **coding** mnemonic is affected. Everything else either fits locally or is cloud-based. Cloud fallback for coding uses `uni-mainz/Qwen3 Coder 30B` (free, code-specialized, ~0.7s latency). If Uni Mainz is rate-limited (20 req/window), switch to `google/gemini-3.5-flash` or `@preset/ros2k-auto`.

On weaker GPUs, Gazebo rendering competes more aggressively for VRAM. The strategy already handles this: if GPU is busy, all cloud fallbacks remain available.

Install script auto-detection: The team tarball install script detects GPU VRAM via `nvidia-smi --query-gpu=memory.total` and sets the coding model accordingly:

GPU VRAM	model (coding default)
>= 20 GB	ollama/qwen2.5-coder:32b (local)
8-19 GB	uni-mainz/Qwen3 Coder 30B (cloud)
< 8 GB	openrouter/@preset/ros2k-auto (cloud)

The `summarize`, `chat`, and `search` agents stay on local 3B/7B regardless — those fit on any GPU. Only the main `coding` model shifts to cloud.

2.3 Ollama Cloud

Ollama Cloud provides access to 19 models via `https://ollama.com/v1` (OpenAI-compatible endpoint), including large-scale models not available

or expensive through other providers. The API key is distributed to all team members. The shared budget is \$10/month — use sparingly for deep think tasks only.

Model	Size	Role	Available elsewhere?
qwen3.5:397b	397B	deep think	Not on OpenRouter; Uni Mainz has Qwen3 235B
kimi-k3	—	deep think	\$3/\$15 on OpenRouter; included in Ollama Cloud budget
gpt-oss:120b	120B	deep think	Free on Uni Mainz; available on Ollama Cloud
glm-5.2	—	(reference)	\$0.10/\$0.30 on OpenRouter

These models are added to the Favorites in opencode as "deep think" entries. The Ollama Cloud API key is included in the team tarball as a literal value (shared, like Uni Mainz).

For non-student team members or external collaborators: These models and the shared API key are NOT distributed outside the team. If you are not a team member and want Ollama Cloud access, create your own account at `ollama.com` — pricing starts at \$10/month for a personal subscription with a limited token budget.

2.4 OpenRouter Presets

Presets are server-side named configurations (`@preset/<slug>`) stored in the OpenRouter workspace. They decouple model routing config from client config — changing a preset on the web UI instantly affects all clients referencing it. Workspace presets are visible to all organization members.

Preset	Model	Cost tier	Allowed models	Provider sort
<code>@preset/ros2k-auto</code>	<code>openrouter/auto</code>	Low	5 free models (see below)	Price

Allowed models for `ros2k-auto` :

- `qwen/qwen3-coder-30b-a3b-instruct` (free, code-specialized, 262K ctx)
- `qwen/qwen3-235b-a22b-2507` (free, 235B, reasoning)
- `deepseek/deepseek-v4-flash` (free, 1M ctx)
- `google/gemma-4-31b-it:free` (free, 262K ctx)

- `nvidia/nemotron-3-nano-30b-a3b:free` (free, 256K ctx)

All 5 models are free with tool calling. **Tool calling** (also known as function calling) is the ability of an LLM to invoke external tools or functions during a conversation — for example, reading a file, executing a bash command, or searching a codebase. `opencode` relies on tool calling for all its operations: the `read`, `write`, `edit`, `bash`, `glob`, and `grep` tools are all invoked via the tool-calling protocol. Models without tool calling support cannot function as `opencode` agents.

The Auto Router classifies each prompt into ~30 task types and selects the best model from the whitelist within the Low cost tier.

2.5 OpenRouter Auto Router

The Auto Router (`openrouter/auto`) is powered by the market: the aggregate spend of millions of OpenRouter users, measured over a trailing 7-day window for each task type. A lightweight classifier assigns each prompt one of ~30 fine-grained task types (e.g. `code:debugging`, `agent:multi_step_planning`, `math`, `research_report`).

Configurable via:

- `cost_tier`: `low` | `medium` | `high` | `xhigh` | `max`
- `allowed_models`: whitelist patterns (e.g. `z-ai/glm-*`, `qwen/*coder*`)
- `excluded_models`: blacklist patterns
- `provider.sort`: `price` | `throughput` | `latency`
- `provider.order`: explicit provider priority list
- `provider.allow_fallbacks`: `false` to pin to ordered providers

Session stickiness: the Auto Router remembers the model a conversation landed on and prefers it on later turns, switching only when the task type changes.

2.6 OpenRouter Provider Routing

Per-request provider selection (applies to all models, not just Auto Router):

Option	Effect
sort: "price"	Cheapest upstream provider
sort: "latency"	Fastest upstream provider
sort: "throughput"	Highest token throughput
order: ["provider_a", ...]	Explicit priority order
allow_fallbacks: false	Pin to ordered providers only
:nitro suffix	Prioritize throughput + priority service tier
:floor suffix	Prioritize cheapest + flex service tier
max_price	Cap cost per request

2.7 Google AI

No auto-routing, no presets, no cost tiers. Model selection is manual.

Free tier has RPD (requests per day) limits. Key models tested:

Model	Context	Tools	Reasoning	Quota status
gemini-3-flash-preview	1M	yes	yes	Exhausted during testing
gemini-3.5-flash	1M	yes	yes (41 thinking tokens)	Available
gemini-3.6-flash	1M	yes	yes (62 thinking tokens)	Available
gemini-3.7-flash	1M	yes	yes	Available (hidden from picker — <code>gemini-flash-latest</code> auto-redirects to newest)
gemini-flash-latest	1M	yes	yes	Available (auto-redirects to newest)
gemini-3.5-flash-lite	1M	yes	no	Available (lighter, faster)

Note: the Google catalog also contains non-LLM models (veo video generation, lyria music generation, image/TTS/live-preview variants, embeddings, gemma-4) — all hidden by the whitelist (see section 5.8).

2.8 Uni Mainz

University-hosted Open WebUI instance. 7 models available. No GLM models. An `auto` router model exists but routes coding tasks to GPT OSS 120B (general-purpose) instead of Qwen3 Coder 30B (code-specialized) — suboptimal for code tasks. Rate limit: 20 requests per ~5 minute window.

Uni Mainz `auto` routing table (probed with 10 task types):

Task type	<code>auto</code> selects	Better choice available
Coding, debugging, code review	GPT OSS 120B	Qwen3 Coder 30B (code-specialized, never selected)
Reasoning, architecture	Qwen3 235B Thinking	Correct
Simple questions, tool calls, summaries	Qwen3 235B VL	Overkill (235B for trivial tasks)

Verdict: Do not use `auto` for coding. Select specific models manually. The Favorites section (see section 5.5 and the preview at the top of this document) solves the manual selection problem: instead of scrolling through a flat list, team members pick from pre-annotated Favorites that already encode the right model for each task type. The mnemonic prefix ("coding", "deep think", etc.) tells you which model to select for which situation — no need to remember model names or capabilities.

3. Task Taxonomy

Based on ROS2K development patterns (502 fast tests, 200-match Gazebo benchmarks, prompt engineering probes, architecture decisions), tasks were classified into 6 complexity levels using 10 probed task types:

Mnemonic	Task types (probed)	Complexity	Frequency
summarize	Title generation, session summaries, compaction	Trivial	Every session start + context-full events
chat	Simple questions, math, general conversation	Low	Occasional
search	Codebase search, file discovery, explore subagent	Low-Medium	Frequent

Mnemonic	Task types (probed)	Complexity	Frequency
coding	Write/debug/review code, short snippets	Medium	Dominant (80% of session)
complex coding	Multi-class design, PID controllers, large refactors	High	Occasional
deep think	Race condition analysis, architecture decisions	Highest	Rare but critical

4. Model Selection Matrix

4.1 Primary + Fallback per Mnemonic

Mnemonic	Primary	Fallback (budget-safe)	Fallback (last resort)	Cost per session
summarize	ollama/qwen2.5:3b (local)	—	—	free
chat	ollama/qwen2.5:3b (local)	uni-mainz/qwen3.6-35b (free)	—	free
search	ollama/qwen2.5-coder:7b (local)	google/gemini-3.5-flash (free, 1M ctx)	—	free
coding	ollama/qwen2.5-coder:32b (local)	uni-mainz/Qwen3 Coder 30B (free)	google/gemini-3.5-flash (free)	free
complex coding	openrouter/z-ai/glm-5.3 (\$0.14/\$0.44 per 1M)	uni-mainz/Qwen3 235B Thinking (free)	uni-mainz/GPT OSS 120B (free)	~\$0.16
deep think	openrouter/z-ai/glm-5.3 (\$0.14/\$0.44 per 1M)	uni-mainz/Qwen3 235B Thinking (free, 235B)	google/gemini-3.5-flash (free, reasoning)	~\$0.16

4.2 Ollama Cloud Extensions

Three additional deep-think models are available via Ollama Cloud (shared \$10/month budget, key distributed to all team members):

Mnemonic	Model	Provider	Role
deep think	qwen3.5:397b	Ollama Cloud	397B — largest available model
deep think	kimi-k3	Ollama Cloud	\$3/\$15 on OpenRouter, included in Ollama Cloud budget
deep think	gpt-oss:120b	Ollama Cloud	120B general-purpose

These are added to the team Favorites as "deep think" entries.

For non-student team members or external collaborators: These models and the shared Ollama Cloud API key are NOT distributed outside the team. If you are not a team member and want Ollama Cloud access, create your own account at ollama.com — pricing starts at \$10/month for a personal subscription with a limited token budget.

4.3 Redundancy Rationale

Each row has 2-3 models from different providers. If the primary is unavailable (GPU busy, budget low, rate-limited), switch to a fallback from a different provider:

Situation	Switch
GPU busy with Gazebo	coding -> uni-mainz/Qwen3 Coder 30B or google/gemini-3.5-flash
OpenRouter budget low	complex coding, deep think -> uni-mainz/Qwen3 235B Thinking (free)
Ollama Cloud budget exhausted	deep think -> openrouter/z-ai/glm-5.3 or uni-mainz/Qwen3 235B Thinking
Uni Mainz rate-limited (20 req/window)	any uni-mainz fallback -> google/gemini-3.5-flash
Google quota exhausted	any google fallback -> uni-mainz/qwen3.6-35b
All paid/free providers exhausted	GitHub Copilot in VSCode (free, GPT-5.4, always available)
Problem exceeds coding capability	coding -> complex coding or deep think (manual /models switch)

4.4 Excluded Models

Model	Price (OpenRouter)	Reason for exclusion
qwen/qwen3.8-max	\$2/\$6 per 1M	Cost-prohibitive for regular use. A typical opencode session (50K prompt + 20K completion) would cost ~\$0.10-0.20 per session — 10x more than GLM 5.3. Reserved for future investigation if budget allows.
moonshotai/kimi-k3	\$3/\$15 per 1M	Cost-prohibitive on OpenRouter (\$0.40/session). However, kimi-k3 IS available via Ollama Cloud (shared \$10/month budget) — included in Favorites as "deep think" for team use.
uni-mainz/auto	Free	Routes coding tasks to GPT OSS 120B (general-purpose 120B) instead of Qwen3 Coder 30B (code-specialized 30B). Probed with 10 task types: auto never selects Qwen3 Coder 30B, uses Qwen3 235B VL for trivial tasks (overkill). Manual selection via Favorites is strictly better.
gemini-3-flash-preview	Free	Quota exhausted on the current Google API key during testing. Google returns "You exceeded your current quota." Use gemini-3.5-flash or gemini-flash-latest instead (same family, quota available).
gemini-2.5-flash , gemini-2.5-pro	Free	Deprecated by Google: "This model is no longer available to new users." Replaced by Gemini 3.x Flash series. Blacklisted in opencode config to prevent confusion.
gemini-2.5-flash-preview-tts , gemini-2.5-pro-preview-tts	Free	Text-to-speech models — not usable for coding/chat. Blacklisted.

4.5 GLM 5.3 — Reasoning Model Notes

GLM 5.3 is a reasoning model: it spends thinking tokens (typically 400-500) before generating content output. `max_tokens` must be ≥ 1000 to get both reasoning AND content. opencode handles this automatically (sets high `max_tokens`). Cost breakdown for a typical deep-think session:

Component	Tokens	Cost
Prompt	~50K	\$0.007

Component	Tokens	Cost
Reasoning (thinking)	~500	\$0.0002
Completion	~20K	\$0.009
Total		~\$0.016

5. OpenCode Configuration

5.1 Config Locations (Layered Merge)

Location	Scope	Precedence
~/.config/opencode/opencode.json	Global (user-wide)	2nd (after remote)
~/.config/opencode/opencode.jsonc	Global (JSONC variant)	Merged with above
<project>/.opencode/opencode.json	Per-project	4th (overrides global)

Configs are merged (not replaced). Later sources override conflicting keys only.

5.2 Config Keys Used

Key	Purpose
model	Default model (coding mnemonic)
small_model	Background tasks (summarize mnemonic)
agent.<name>.model	Per-agent model override (7 built-in agents)
provider.<id>.whitelist	Show only curated models in /models picker
provider.<id>.blacklist	Hide specific models from picker
provider.<id>.models.<model>.name	Annotated display name in picker
compaction.prune	Drop old tool outputs from context (token savings)
tool_output.max_lines	Truncate large tool outputs (token savings)
tool_output.max_bytes	Truncate large tool outputs (token savings)
provider.<id>.options.chunkTimeout	Abort stalled paid requests

5.3 Agent Model Assignment

opencode has 7 built-in agents that accept a `model` override:

Agent	Mnemonic	Model	Rationale
<code>build</code>	<code>coding</code>	(inherits global)	Main coding agent — uses 32B
<code>plan</code>	<code>search</code>	<code>ollama/qwen2.5-coder:7b</code>	Planning needs reasoning, not code gen
<code>explore</code>	<code>search</code>	<code>ollama/qwen2.5-coder:7b</code>	Codebase search — fast + code-aware
<code>title</code>	<code>summarize</code>	<code>ollama/qwen2.5:3b</code>	1-line title — quality irrelevant
<code>summary</code>	<code>summarize</code>	<code>ollama/qwen2.5:3b</code>	Session summary — quality irrelevant
<code>compaction</code>	<code>summarize</code>	<code>ollama/qwen2.5:3b</code>	Context summarization — cheap model
<code>general</code>	<code>coding</code>	(inherits global)	General-purpose agent

If unset, all agents inherit the global `model` — wasting the 32B on trivial tasks like title generation.

5.4 Cost Optimization Levers

Lever	Config key	Estimated token savings
<code>compaction.prune: true</code>	Drop old tool outputs	~30-50% per long session
<code>tool_output.max_lines: 500</code>	Truncate bash/file outputs	~75% on large outputs
<code>tool_output.max_bytes: 20480</code>	Truncate bash/file outputs	~60% on large outputs
<code>agent.title/summary/compaction -> 3B</code>	Background tasks on cheap model	~95% vs 32B for these tasks
<code>agent.explore -> 7B</code>	Search on cheap model	~85% vs 32B for search

5.5 Model Picker: Favorites and Recent

opencode stores model state in `~/.local/state/opencode/model.json`
(XDG **state** dir — confirmed in the opencode 1.18.23 embedded store code,

`XDG_STATE_HOME + "model.json"` ; verified by live mtime updates while the TUI runs):

```
{
  "recent": [{"providerID": "...", "modelID": "..."}, ...],
  "favorite": [{"providerID": "...", "modelID": "..."}, ...],
  "variant": {}
}
```

The `/models` picker has 3 sections in this order:

1. **Favorites** — from `model.json` `favorite` array
2. **Recent** — from `model.json` `recent` array, excluding favorites
3. **All models** — grouped by provider, excluding favorites and recent

Favorites are keyed on `providerID:modelID` — one model = one favorite entry. Mnemonics that share the same model are combined in the annotated name (e.g. "summarize, chat" or "complex coding, deep think").

Favorites persist across opencode updates. The `model.json` file is never automatically pruned. If a model disappears from the catalog (e.g. Google deprecates a model), it stops showing in Favorites but the entry remains in `model.json`. If the model reappears later, it shows again automatically. Pre-populating `model.json` in the team tarball gives every team member the same Favorites on first launch.

Editing `model.json` by hand — two gotchas (learned 2026-08-25):

1. The TUI loads `state/model.json` at startup and keeps an in-memory copy. Hand-edits made while a TUI session is running are silently overwritten on the next model switch. Always edit with no opencode TUI running, then start the TUI.
2. The `opencode models` CLI validates only provider whitelists against the catalog — it does NOT resolve the Favorites entries. A wrong-path or stale `model.json` passes every CLI check while `/models` shows a broken list. Verify Favorites in the TUI picker itself.

5.6 Model Picker: Annotated Names

opencode has no favorites, tags, grouping, or annotation fields beyond the `name` field. The `name` is the only user-configurable display text. Workaround: embed the mnemonic + model name + provider in `name` and use `whitelist/blacklist` to control visibility.

Format: <mnemonic> – <model name> – <provider display name>

Example: "coding – Qwen 2.5 Coder 32B – Ollama"

5.7 Model Picker: Sorting

The `/models` picker sorts by 3 layers:

- 1. **Current model first** — whatever model is active appears at top
- 2. **Hardcoded recommended list** — opencode's built-in priority array (not user-configurable)
- 3. **Alphabetical by name** — fallback for models not in the priority array

Models are grouped by provider. Provider group order is also hardcoded (not user-configurable). The annotated `name` affects sort position within a provider group for models not in opencode's built-in recommended list.

5.8 Provider Filtering Strategy

Provider	Use whitelist or blacklist ?	Why
Ollama	whitelist (3 models)	Only 3 models pulled; hiding the rest is correct
OpenRouter	whitelist (2 entries)	Hundreds of models; whitelist is essential
Uni Mainz	whitelist (4 models)	Only 7 models exist; whitelist hides bad ones (auto, bge-m3)
Google	whitelist (4 models)	v1.1 used a blacklist for deprecated 2.5 models, but the catalog exploded (veo video gen, lyria music gen, image/TTS/live/embedding models, gemma-4, gemini-3.7-flash, deep-research) — 30+ entries. Whitelisted: gemini-3.5-flash (Favorite), gemini-3.6-flash, gemini-flash-latest, gemini-3.1-pro-preview. New Google models must be added manually.
opencode (Zen free)	whitelist (empty = hidden)	opencode's built-in free-tier provider (big-pickle, muse-spark-1.2-contributor-free, nemotron-3-ultra-free, ...) is not part of the strategy. An empty whitelist hides the entire provider.

Change log v1.1 → v1.2 (2026-08-25): `/models` showed 50+ models instead of the proposed 16 — the Google blacklist was insufficient (catalog explosion) and the built-in `opencode` Zen provider was unfiltered. Fixed by switching Google to a whitelist and hiding the Zen provider. `model.json` Favorites were already correct and unchanged.

Change log v1.2 → v1.3 (2026-08-25): `/models` still showed only 1 entry after the v1.2 fix. Root cause: the curated Favorites were pre-populated into `~/.local/share/opencode/model.json` (XDG data dir), but `opencode`'s TUI reads and writes `~/.local/state/opencode/model.json` (XDG state dir). The stale state file held 9 old favorites of which 8 pointed at the now-hidden Zen provider and 1 at delisted `gemini-3.7-flash` — leaving exactly one survivor. Fixed by rewriting the state file with the curated 11 (favorite + recent, variant map preserved) and archiving the wrong-path file as `model.json.bak-20260825-wrongpath`. Additionally: `tool_call: true` added to the 3 Ollama model definitions (was missing — asymmetric with `uni-mainz`; the local Qwen models declare the `tools` capability in Ollama but `opencode` did not treat them as tool-capable).

6. Cross-Tool Applicability

6.1 No Shared Standard

There is no cross-tool standard for:

- Agent model assignment
- Background-task routing (`small_model`)
- Compaction pruning
- Tool output truncation
- Model picker annotations

Each tool (`opencode`, `Continue.dev`, `VSCode Copilot`, `Cline`) invented its own config schema. Configs are NOT interchangeable.

6.2 Provider APIs Are Shared

All tools share the underlying LLM provider APIs (OpenAI-compatible `/v1/chat/completions`). `Ollama`, `OpenRouter`, `Google AI`, and `Uni Mainz` endpoints work regardless of which tool consumes them. Only the config wrapper differs per tool.

6.3 OpenRouter Presets Are Tool-Agnostic

OpenRouter presets (@preset/<slug>) are server-side. Any tool that sends model: "@preset/ros2k-auto" to the OpenRouter API gets the preset's config applied. This makes presets the most portable config layer — they work across opencode, Continue, raw API calls, and any OpenAI-compatible client.

7. Testing Methodology

7.1 Model Quality Probe

Each candidate model was tested with:

- 1. **Code generation** — "Write a Python ROS2 node that subscribes to /gazebo/model_states, extracts 2D positions, writes flat JSON via os.replace at 10Hz"
- 2. **Tool calling** — "What is the weather in Berlin? Use the get_weather tool."
- 3. **Reasoning** — "Analyze race conditions in Python file polling on tmpfs"
- 4. **Latency** — timed curl calls for simple and complex prompts

7.2 Uni Mainz Auto Router Probe

10 task types sent to uni-mainz/auto . Response model field inspected to determine routing:

Task	Auto selected
Coding (ROS2 node)	GPT OSS 120B
Debugging	GPT OSS 120B
Code review	GPT OSS 120B
Math	GPT OSS 120B
Reasoning (race conditions)	Qwen3 235B Thinking
Architecture (path planner)	Qwen3 235B Thinking
Complex coding (PID controller)	Qwen3 235B Thinking
Simple question (2+2)	Qwen3 235B VL
Tool calling	Qwen3 235B VL
Summary	Qwen3 235B VL

7.3 OpenRouter Auto Router Cost Tier Probe

5 cost tiers tested with a coding prompt:

Cost tier	Model selected	Cost per call
low	deepseek-v4-flash-0731	\$0.00009
medium	z-ai/glm-5.2	\$0.002
high	openai/gpt-5.6-sol	\$0.0016
xhigh	moonshotai/kimi-k3	\$0.005
max	anthropic/claude-opus-5	\$0.008

7.4 Google Model Availability Probe

6 Google Flash models tested for availability:

Model	Available?	Reasoning tokens
gemini-3-flash-preview	No (quota exhausted)	—
gemini-3.5-flash	Yes	41
gemini-3.6-flash	Yes	62
gemini-flash-latest	Yes	7
gemini-3.5-flash-lite	Yes	0 (no reasoning)
gemini-flash-lite-latest	Yes	0 (no reasoning)

8. Team Favorites

The team tarball includes the same Favorites list for all team members:

```
summarize, chat – Qwen 2.5 3B – Ollama
search – Qwen 2.5 Coder 7B – Ollama
coding – Qwen 2.5 Coder 32B – Ollama
coding (fallback) – Qwen3 Coder 30B – Uni Mainz
auto – @preset/ros2k-auto router – OpenRouter
complex coding, deep think – GLM 5.3 – OpenRouter
deep think – Qwen 3.5 397B – Ollama Cloud
deep think – Kimi K3 – Ollama Cloud
deep think – GPT OSS 120B – Ollama Cloud
deep think (fallback) – Qwen3 235B Thinking – Uni Mainz
fallback – Gemini 3.5 Flash – Google
```

8.1 API Key Distribution

Key	In team tarball?	Distribution
Ollama (local)	N/A (no key needed)	—
Uni Mainz	Yes (literal value)	Shared, free for all
Ollama Cloud	Yes (literal value)	Shared, \$10/month budget for all team members
Google	No (placeholder)	Each member gets own at ai.google.dev
OpenRouter	No (placeholder)	Each member creates own after org invite
GitHub Copilot	N/A (VSCode-managed)	Team members log in via VSCode

9. Other Recommendations

9.1 GitHub Copilot

Free academic quota (`free_faculty_quota`) available to all 10 team members. Uses GitHub's own API infrastructure — not OpenRouter, not Google, not Uni Mainz. No cost to the team.

VSCode extension (Copilot Chat v0.56.0): 3 built-in agents:

Agent	Description	Model options
Ask	Read-only Q&A, code explanation, debugging guidance	Copilot default (GPT-5.4)
Explore	Fast read-only codebase exploration subagent	Claude Haiku 4.5, Gemini 3 Flash, Auto
Plan	Research and outline multi-step plans	Copilot default (GPT-5.4)

Copilot CLI: Uses `copilotcli/gpt-5.4` (stored in VSCode state DB). Requires `gh auth login` to function from terminal.

Copilot as ultimate fallback: If OpenRouter budget is exhausted, Uni Mainz is rate-limited, Google quota is hit, and the local GPU is busy with Gazebo — Copilot Chat in VSCode is still available, free, with GPT-5.4. This makes VSCode + Copilot a reliable parallel development environment alongside opencode.

9.2 Two Development Environments

Environment	Tool	Primary use	Models
Terminal	opencode	CLI-based coding, agents, tools	Ollama + OpenRouter + Uni Mainz + Google + Ollama Cloud
VSCode	Copilot Chat + Continue	IDE-based coding, inline completion, chat	GitHub Copilot (GPT-5.4 etc.) + Continue models

GitHub Copilot is free for all 10 team members via academic quota. It serves as the ultimate fallback when all other providers are unavailable. The 3 Copilot agents (Ask, Explore, Plan) map to our mnemonics: Ask = chat, Explore = search, Plan = deep think.

10. Open Items

1. **OpenRouter org invitations** — admin must invite team members via web UI at `openrouter.ai/settings/organization-members`. Members accept email invite, create account, create API key at `openrouter.ai/workspaces/default/keys`.
2. **Uni Mainz October retirement** — 4 of 7 models retire. Post-October fallback chain will need adjustment. `qwen3.6-35b` and `minimax-m3` stay.
3. **Google quota** — `gemini-3-flash-preview` exhausted. `gemini-3.5-flash` and `gemini-flash-latest` still available. May need billing setup for higher RPD if team usage grows.
4. **Team tarball distribution** — separate channel (not git). Created after team meeting feedback is incorporated.
5. **Copilot CLI setup** — `gh auth login` required for terminal use. Team members do this manually (not in tarball install script).

11. Automatic vs Manual Model Routing

opencode's internal routing mechanisms are **automatic** — they fire without any user interaction after restart. The OpenRouter Auto Router and escalation models require **manual selection** via `/models`.

11.1 Automatic (no user interaction)

Mechanism	When it fires	Model	Config key
Coding default	Every prompt you type	ollama/qwen2.5-coder:32b	<code>model</code>
Title generation	Session start	ollama/qwen2.5:3b	<code>small_model</code> or <code>agent.title</code>
Session summary	Context compaction	ollama/qwen2.5:3b	<code>agent.summary</code>
Context compaction	Context window fills	ollama/qwen2.5:3b	<code>agent.compaction</code>
Codebase search (explore)	Explore subagent runs	ollama/qwen2.5-coder:7b	<code>agent.explore</code>
Planning mode	Plan agent activated	ollama/qwen2.5-coder:7b	<code>agent.plan</code>
Tool output truncation	Every tool call	—	<code>tool_output.max_lines/max_bytes</code>
Compaction pruning	Context window fills	—	<code>compaction.prune</code>
Provider filtering	<code>/models</code> picker display	—	<code>whitelist</code> / <code>blacklist</code>
Favorites display	<code>/models</code> picker	—	<code>model.json</code> <code>favorite</code> array

These mechanisms handle ~80% of daily work (coding, search, background tasks). Zero manual interaction needed after restart.

11.2 Manual selection (via `/models` -> Favorite)

Model	Favorite position	When to select
@preset/ros2k-auto router (OpenRouter)	#5	GPU busy with Gazebo, or want free cloud routing without choosing a specific model
GLM 5.3 (OpenRouter)	#6	Race conditions, architecture decisions, complex coding — needs reasoning

Model	Favorite position	When to select
Qwen 3.5 397B (Ollama Cloud)	#7	Deep think when OpenRouter budget is low — 397B largest available
Kimi K3 (Ollama Cloud)	#8	Deep think alternative (expensive on OpenRouter, included in Ollama Cloud budget)
GPT OSS 120B (Ollama Cloud)	#9	Deep think alternative — 120B general-purpose
Qwen3 235B Thinking (Uni Mainz)	#10	Deep think fallback — free, 235B, reasoning model
Gemini 3.5 Flash (Google)	#11	Universal fallback — free, 1M context, reasoning
Qwen3 Coder 30B (Uni Mainz)	#4	Coding fallback when local GPU busy — free, code-specialized
Qwen3.6 35B (Uni Mainz)	(in All models)	Chat fallback — free, general-purpose

To switch: type `/models` , select the desired Favorite. The current session switches to that model. Switching back: `/models -> select "coding — Qwen 2.5 Coder 32B — Ollama"` (Favorite #3).

11.3 How the Auto Router works after manual selection

When you select `@preset/ros2k-auto` (Favorite #5):

1. `opencode` sends `model: "@preset/ros2k-auto"` to OpenRouter
2. OpenRouter resolves the preset server-side
3. The Auto Router classifies your prompt into a task type (e.g. `code:debugging` , `agent:multi_step_planning`)
4. It selects the best model from the 5-model whitelist within the Low cost tier, sorted by price
5. The selected model handles the request
6. The response `model` field shows which model was actually used — `opencode` displays this in the response metadata
7. Session stickiness: the Auto Router remembers the model and prefers it on subsequent turns, switching only when the task type changes

The Auto Router is NOT the default model — the default is the local 32B coder (free, no network latency). The Auto Router is a manual

fallback for when the local GPU is busy or when you want free cloud routing without choosing a specific model.